Scenario-Based AWS Interview Q&A
AWS Architecture & Scenario-Based — Important Interview Questions & Answers

1.How does a user request reach an EC2 application?
A user request can first reach Route 53 for DNS resolution and then CloudFront or an Application Load Balancer depending on the architecture.
The Load Balancer distributes the request to healthy EC2 instances running inside the application environment.
Simple ga gurthupettuko:
User → Route 53 → ALB → EC2.
Eng: The request is routed through DNS and the load balancer to a healthy EC2 instance.

2.How do you design a highly available application in AWS?
I would deploy the application across multiple Availability Zones and use an Application Load Balancer to distribute traffic.
I would use Auto Scaling to maintain the required number of instances and a Multi-AZ database for database availability.
Simple ga gurthupettuko:
Multi-AZ + ALB + Auto Scaling + Multi-AZ DB = High Availability.
Eng: I would use multiple Availability Zones, load balancing, Auto Scaling, and a highly available database.

3.What is a Public EC2 and Private EC2 architecture?
A public EC2 instance is placed in a public subnet and can have a route to the Internet Gateway.
A private EC2 instance is placed in a private subnet and is normally accessed through internal services or controlled outbound connectivity.
Simple ga gurthupettuko:
Public EC2 internet-facing, Private EC2 direct internet access lekunda internal ga untundi.
Eng: Public EC2 can be internet-facing, while private EC2 is kept inside a private network.

4.How would you design an application using ALB, ASG, and EC2?
I would place an ALB in public subnets and EC2 instances in private subnets across multiple Availability Zones.
The Auto Scaling Group would maintain the required EC2 instances, while the ALB distributes traffic only to healthy instances.
Simple ga gurthupettuko:
Internet → ALB → Private EC2 → Database, ASG instances ni manage chestundi.
Eng: ALB distributes traffic to EC2 instances managed by Auto Scaling across multiple Availability Zones.

5.How does a Private EC2 instance access the Internet?
A private EC2 instance can send outbound Internet traffic through a NAT Gateway placed in a public subnet.
The private subnet route table points Internet-bound traffic to the NAT Gateway.
Simple ga gurthupettuko:
Private EC2 → Private Route Table → NAT Gateway → Internet.
Eng: A private EC2 instance can access the Internet through a NAT Gateway.

6.How would you secure an AWS application?
I would keep application servers in private subnets and expose only required public components such as the load balancer.
I would use IAM with least privilege, Security Groups, encryption, HTTPS, logging, monitoring, and services such as WAF where required.
Simple ga gurthupettuko:
Private Subnets + IAM + SG + HTTPS + Encryption + WAF = Secure Architecture.
Eng: I would use private networking, least privilege, security controls, encryption, and monitoring.

7.How would you troubleshoot an unreachable EC2 instance?
I would first check the instance state and then verify the subnet route table, Internet Gateway or NAT configuration, Security Group, and Network ACL.
I would also check the instance operating system, application service, and system logs if network connectivity is available.
Simple ga gurthupettuko:
EC2 State → Route Table → IGW/NAT → SG → NACL → OS/Application check chestham.
Eng: I would check the instance, routing, security groups, network ACLs, and operating system.

8.How would you reduce AWS cost?
I would identify unused or underutilized resources using monitoring and cost tools.
Then I would right-size resources, use appropriate pricing options such as Savings Plans or Reserved Instances where suitable, and remove unnecessary resources.
Simple ga gurthupettuko:
Unused resources remove + Right-size + suitable pricing option = Cost Optimization.
Eng: I would reduce cost by removing unused resources, right-sizing, and using suitable pricing options.

9.How would you design Disaster Recovery in AWS?
I would first identify the application's RTO and RPO requirements and then choose an appropriate DR strategy.
I would use backups, replication, Multi-Region architecture, or another suitable recovery approach based on the business requirement.
Simple ga gurthupettuko:
RTO/RPO decide chesi → Backup/Replication → DR Region → Failover plan.
Eng: I would design DR based on RTO, RPO, backups, replication, and failover requirements.

10.What is the difference between Backup and Disaster Recovery?
Backup means creating copies of data so that it can be restored after data loss or corruption.
Disaster Recovery is the complete strategy for restoring applications, infrastructure, and data after a major failure.
Simple ga gurthupettuko:
Backup = Data copy, DR = Complete application recovery plan.
Eng: Backup protects data, while Disaster Recovery focuses on restoring the complete environment.

11.What is RTO?
RTO stands for Recovery Time Objective.
It defines the maximum acceptable time an application can take to recover after a failure.
Simple ga gurthupettuko:
RTO ante failure tarvata application entha time lo recover avvali ani define chestundi.
Eng: RTO defines the maximum acceptable recovery time after a failure.

12.What is RPO?
RPO stands for Recovery Point Objective.
It defines the maximum acceptable amount of data loss measured in time.
Simple ga gurthupettuko:
RPO ante maximum entha time data loss tolerate cheyyagalamo define chestundi.
Eng: RPO defines the maximum acceptable amount of data loss in time.

13.What is the difference between Multi-AZ and Multi-Region?
Multi-AZ means deploying resources across multiple Availability Zones within the same AWS Region.
Multi-Region means deploying resources across different AWS Regions and is commonly used for higher resilience and disaster recovery.
Simple ga gurthupettuko:
Multi-AZ = Same Region, multiple AZs. Multi-Region = Different AWS Regions.
Eng: Multi-AZ uses multiple Availability Zones in one Region, while Multi-Region uses multiple Regions.

14.How would you deploy an application with zero or minimal downtime?
I would use a deployment strategy such as Blue/Green or Rolling deployment depending on the application.
I would deploy and test the new version separately and gradually or safely shift traffic after validation.
Simple ga gurthupettuko:
New Version → Test → Traffic Shift → Old Version remove.
Eng: I would use Blue/Green or Rolling deployment to minimize application downtime.

15.How would you troubleshoot an application that is suddenly down?
I would first check the application health and monitoring alerts.
Then I would check the load balancer, EC2 instances, Auto Scaling, security groups, network connectivity, database, and application logs to identify the failure.
Simple ga gurthupettuko:
Monitoring → ALB → EC2/ASG → Network/SG → DB → Logs check chestham.
Eng: I would check monitoring, load balancing, compute, networking, database, and logs.

16.What is a common highly available AWS architecture?
A common architecture uses Route 53 for DNS, an Application Load Balancer across multiple Availability Zones, EC2 instances managed by Auto Scaling, and an RDS Multi-AZ database.
This architecture reduces single points of failure and improves application availability.
Simple ga gurthupettuko:
Route 53 → ALB → ASG/EC2 → RDS Multi-AZ.
Eng: A highly available architecture uses DNS, load balancing, Auto Scaling, and a Multi-AZ database.

17.How would you identify a single point of failure in AWS architecture?
I would check whether any critical component exists in only one Availability Zone or has only one instance or connection.
I would then introduce redundancy using multiple AZs, load balancing, Auto Scaling, or other suitable high-availability components.
Simple ga gurthupettuko:
One critical resource only one place lo unte Single Point of Failure.
Eng: I would identify single-instance or single-AZ dependencies and add redundancy.

18.What is the most important thing to remember in AWS architecture interviews?
I would focus on Availability, Security, Scalability, Performance, Cost, and Disaster Recovery.
The architecture should avoid single points of failure and use the appropriate AWS services based on the application requirements.
Simple ga gurthupettuko:
AWS Architecture = HA + Security + Scalability + Performance + Cost + DR.
Eng: A good AWS architecture should be highly available, secure, scalable, cost-effective, and recoverable.